SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI – 602105

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Dataset Intégration Task

Course Code:	DSA0502	**Course Title:**	Query Processing for Data Science
CO Assessed:	CO3	**Assessment:**	Tool 3 - Dataset Intégration Task
Weightage:		**Total Marks:**	
CO3 Statement: Identify and clean inconsistencies in data by handling missing values, duplicates, outliers, formatting issues, and applying techniques like fuzzy matching and regex. (BL5 and BL6).			
Student Name:	Sania Herbert	**Reg. No.:**	192324062
Section / Batch:	2023/AI&DS	**Date of Submission:**	14.8.26
Faculty In-charge:	**Dr. B.Shyamala Gowri**	**Project Mode:**	Individual Submission

Code of Conduct

I Sania Herbert(192324062) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: Sania

Dataset Intégration Task

Question 1 – Customer Dataset Integration

Three customer datasets from **Online, Retail, and Support systems** contain inconsistent names, phone numbers, email addresses, dates, duplicate records, and missing values.

Task: Using Python/Pandas, clean and integrate the datasets by identifying invalid values, formatting data, removing duplicates, applying fuzzy matching and RegEx validation, standardizing fields, and saving the final master dataset in CSV and JSON formats. Create a reusable cleanup script and test it with a new dataset.

Dataset 1: Online Customers

Customer_ID	Customer_Name	Phone_Number	Email_Address	Date_of_Birth
C001	Sania Herbert	9876543210	sania.herbert@gmail.com	12/06/2003
C002	Rahul Sharma	98765-43211	rahul.sharma@gmail.com	05-11-2002
C003	Priya Nair	919876543212	priya.nair@gmail.com	2001/08/19
C004	Arun Kumar	9876543213	arun.kumar@gmail	23/03/2000
C005	Meena Joseph		meena.joseph@gmail.com	15-09-2001
C006	Vikram Singh	9876543215	vikram.singh@gmail.com	01/01/2003

Dataset 2: Retail Customers

Retail_ID	Customer	Mobile	Email_ID	Birth_Date
R101	Sania Herbert	+91 9876543210	SANIA.HERBERT@GMAIL.COM	12-06-2003
R102	Rahul Sharm	9876543211	rahul.sharma@gmail.com	05/11/2002
R103	Priya Nair	9876543212	priya.nair@gmail.com	19-08-2001
R104	Arun Kumar	9876543213	arun.kumar@gmail.com	23-03-2000
R105	Meena Joseph	9876543214	meena.joseph@gmail.com	
R106	Vikram Singh	9876543215	vikram.singh@gmail.com	01-01-2003

Dataset 3: Support Customers

Support_ID	Customer_Name	Contact_Number	Email_ID	Birthdate
S201	Sania Herbert	09876543210	sania.herbert@gmail.com	2023-06-12
S202	Rahul Sharma	+91-9876543211	rahul.sharma@gmail.com	2022-11-05
S203	Priya Nair	9876543212	priya.nair@gmail.com	19/08/2001
S204	Arun Kumar	9876543213	arun.kumar@gmail.com	23/03/2000
S205	Meena Joseph	9876543214	invalid-email	15/09/2001
S206	Vikram Singh	9876543215	vikram.singh@gmail.com	01/01/2003

C: > Users > DELL > task integration.py > ...

```python
import pandas as pd
import numpy as np
import re
from rapidfuzz import fuzz
# ------------------------------------------------------------
# 1. LOAD EXCEL DATASET
# ------------------------------------------------------------
file_path = r"C:\Users\DELL\Downloads\Q1_Customer_Dataset.xlsx"
online = pd.read_excel(file_path, sheet_name="Online")
retail = pd.read_excel(file_path, sheet_name="Retail")
support = pd.read_excel(file_path, sheet_name="Support")
# ------------------------------------------------------------
# 2. STANDARDIZE COLUMN NAMES
# ------------------------------------------------------------
online.columns = (
    online.columns
    .str.strip()
    .str.lower()
    .str.replace(" ", "_")
)
retail.columns = (
    retail.columns
    .str.strip()
    .str.lower()
    .str.replace(" ", "_")
)
support.columns = (
    support.columns
    .str.strip()
    .str.lower()
    .str.replace(" ", "_")
)
```

```python
# ------------------------------------------------------------
# 3. RENAME DIFFERENT COLUMN NAMES
# ------------------------------------------------------------
online = online.rename(columns={
    "customer_name": "name",
    "phone_number": "phone",
    "email_address": "email",
    "date_of_birth": "dob"
})
retail = retail.rename(columns={
    "customer": "name",
    "mobile": "phone",
    "email_id": "email",
    "birth_date": "dob"
})
support = support.rename(columns={
    "customer_name": "name",
    "contact_number": "phone",
    "email id": "email",
    "email_id": "email",
    "birthdate": "dob"
})
# ------------------------------------------------------------
# 4. FUNCTION FOR CLEANING NAME
# ------------------------------------------------------------
def clean_name(name):
    if pd.isna(name):
        return np.nan
    name = str(name).strip()
    # Remove special characters
    name = re.sub(r"[^A-Za-z ]", "", name)
    # Remove extra spaces
    name = re.sub(r"\s+", " ", name)
    # Convert to title case
    return name.title()
```

```python
# ------------------------------------------------------------
# 5. PHONE VALIDATION
# ------------------------------------------------------------
def clean_phone(phone):
    if pd.isna(phone):
        return np.nan
    # Keep only numbers
    phone = re.sub(r"\D", "", str(phone))
    # 10-digit Indian number
    if len(phone) == 10:
        return phone
    # 91 + 10 digits
    if len(phone) == 12 and phone.startswith("91"):
        return phone[2:]
    return np.nan
```

```python
# ------------------------------------------------------------
# 6. EMAIL VALIDATION USING REGEX
# ------------------------------------------------------------
def clean_email(email):
    if pd.isna(email):
        return np.nan
    email = str(email).strip().lower()
    pattern = r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[A-Za-z]{2,}$"
    if re.match(pattern, email):
        return email
    return np.nan
```

```python
# ------------------------------------------------------------
# 7. DATE STANDARDIZATION
# ------------------------------------------------------------
def clean_date(date):
    if pd.isna(date):
        return pd.NaT
    return pd.to_datetime(
        date,
        errors="coerce",
        dayfirst=True
    )
# ------------------------------------------------------------
```

```python
# ------------------------------------------------------------
# 8. CLEAN EACH DATASET
# ------------------------------------------------------------
for df in [online, retail, support]:
    df["name"] = df["name"].apply(clean_name)
    df["phone"] = df["phone"].apply(clean_phone)
    df["email"] = df["email"].apply(clean_email)
    if "dob" in df.columns:
        df["dob"] = df["dob"].apply(clean_date)
# ------------------------------------------------------------
# 9. REMOVE EXACT DUPLICATES
# ------------------------------------------------------------
online = online.drop_duplicates()
retail = retail.drop_duplicates()
support = support.drop_duplicates()
```

```python
# 10. COMBINE ALL DATASETS
# ------------------------------------------------------------
customers = pd.concat(
    [online, retail, support],
    ignore_index=True
)
# ------------------------------------------------------------
# 11. FUZZY MATCHING FOR CUSTOMER NAMES
# ------------------------------------------------------------
def normalize_name(name):
    if pd.isna(name):
        return ""
    return re.sub(
        r"\s+",
        "",
        name.lower()
    )
customers["name_key"] = customers[
```

```python
customers["name_key"] = customers[
    "name"
].apply(normalize_name)
master_records = []
for _, row in customers.iterrows():
    matched = False
    for master in master_records:
        score = fuzz.ratio(
            row["name_key"],
            master["name_key"]
        )
        # Similarity >= 85%
        if score >= 85:
            # Fill missing phone
            if (
                pd.isna(master["phone"])
                and not pd.isna(row["phone"])
            ):
                master["phone"] = row["phone"]
```

```python
                master["phone"] = row["phone"]
            # Fill missing email
            if (
                pd.isna(master["email"])
                and not pd.isna(row["email"])
            ):
                master["email"] = row["email"]
            matched = True
            break
    if not matched:
        master_records.append(
            row.to_dict()
        )
# ------------------------------------------------------------
# 12. CREATE MASTER DATAFRAME
# ------------------------------------------------------------
master = pd.DataFrame(
    master_records
```

```
    )
# ------------------------------------------------------------
# 12. CREATE MASTER DATAFRAME
# ------------------------------------------------------------
master = pd.DataFrame(
    master_records
)
# ------------------------------------------------------------
# 13. REMOVE DUPLICATES USING EMAIL
# ------------------------------------------------------------
master = master.drop_duplicates(
    subset=["email"],
    keep="first"
)
# ------------------------------------------------------------
# 14. REMOVE DUPLICATES USING PHONE
# ------------------------------------------------------------
master = master.drop_duplicates(
    subset=["phone"],
```

```
master = master.drop_duplicates(
    subset=["phone"],
    keep="first"
)
# ------------------------------------------------------------
# 15. HANDLE MISSING VALUES
# ------------------------------------------------------------
master["name"] = master[
    "name"
].fillna("Unknown")
master["phone"] = master[
    "phone"
].fillna("Not Available")
master["email"] = master[
    "email"
].fillna("Not Available")
# ------------------------------------------------------------
# 16. REMOVE TEMPORARY COLUMN
```

```
# 16. REMOVE TEMPORARY COLUMN
# ------------------------------------------------------------
if "name_key" in master.columns:
    master.drop(
        columns=["name_key"],
        inplace=True
    )
# ------------------------------------------------------------
# 17. SAVE FINAL MASTER DATASET AS CSV
# ------------------------------------------------------------
master.to_csv(
    "customer_master.csv",
    index=False
)
# ------------------------------------------------------------
# 18. SAVE FINAL MASTER DATASET AS JSON
# ------------------------------------------------------------
master.to_json(
```

```
# ------------------------------------------------------------
# 18. SAVE FINAL MASTER DATASET AS JSON
# ------------------------------------------------------------
master.to_json(
    "customer_master.json",
    orient="records",
    indent=4,
    date_format="iso"
)
# ------------------------------------------------------------
# 19. DISPLAY RESULTS
# ------------------------------------------------------------
print("\n======================================")
print("CUSTOMER DATA INTEGRATION COMPLETED")
print("======================================")
print("\nOnline Records:", len(online))
print("Retail Records:", len(retail))
print("Support Records:", len(support))
print("\nFinal Master Dataset Shape:")
print(master.shape)
print("\nFinal Master Dataset:")
print(master)
print("\nFiles Created:")
print("customer_master.csv")
print("customer_master.json")
```

OUTPUT:

```
CUSTOMER DATA INTEGRATION COMPLETED
======================================

Online Records: 6
Retail Records: 6
Support Records: 6


Final Master Dataset Shape:
(6, 7)
```

Final Master Dataset:

	customer_id	name	phone	email	dob	retail_id	support_id
0	C001	Sania Herbert	9876543210	sania.herbert@gmail.com	2003-06-12	NaN	NaN
1	C002	Rahul Sharma	9876543211	rahul.sharma@gmail.com	2002-11-05	NaN	NaN
2	C003	Priya Nair	9876543212	priya.nair@gmail.com	2001-08-19	NaN	NaN
3	C004	Arun Kumar	9876543213	arun.kumar@gmail.com	2000-03-23	NaN	NaN
4	C005	Meena Joseph	9876543214	meena.joseph@gmail.com	2001-09-15	NaN	NaN
5	C006	Vikram Singh	9876543215	vikram.singh@gmail.com	2003-01-01	NaN	NaN

Files Created:

customer_master.csv

customer_master.json

Question 4 – Student Dataset Integration

A college maintains student information in **Admission, Examination, and Attendance datasets**. Student names have spelling variations, register numbers have different formats, dates are inconsistent, email addresses may be invalid, and duplicate student records exist.

Task: Using Python/Pandas, clean and integrate the datasets by applying **data formatting, missing-value handling, RegEx matching, fuzzy matching, duplicate identification, normalization, and standardization**. Create a master student dataset and write a reusable Python cleanup script. Test the script with a new set of student records.

Dataset 1: Admission

Admission_ID	Register_Number	Student_Name	Email_Address	Department	Admission_Date
ADM001	23AIDS001	Sania Herbert	sania.herbert@gmail.com	AI & DS	15/06/2023
ADM002	23AIDS002	Rahul Kumar	rahul.kumar@gmail.com	AI & DS	16-06-2023
ADM003	23AIDS003	Priya Nair	priya.nair@gmail.com	CSE	2023/06/17
ADM004	23AIDS004	Arun Raj	arun.raj@gmail.com	ECE	18/06/2023
ADM005	23AIDS005	Meena Joseph	meena.joseph@gmail.com	AI & DS	19-06-2023
ADM006	23AIDS006	Vikram Singh	vikram.singh@gmail.com	CSE	20/06/2023

Dataset 2: Examination

Exam_ID	Registration_No	Student	Email_ID	Marks	Exam_Date
EX001	23-AIDS-001	Sania Herbert	sania.herbert@gmail.com	88	10/04/2026
EX002	23AIDS002	Rahul Kumar	rahul.kumar@gmail.com	76	2026-04-10
EX003	23AIDS003	Priya Nair	priya.nair@gmail.com	91	11-04-2026
EX004	23AIDS004	Arun Raaj	arun.raj@gmail.com	67	12/04/2026
EX005	23AIDS005	Meena Joseph	invalid-email	84	2026/04/12
EX006	23AIDS006	Vikram Singh	vikram.singh@gmail.com	72	13-04-2026

Dataset 3: Attendance

Attendance_ID	Registration_Number	Student_Name	Attendance_Percent	Department_Name
ATT001	23AIDS001	Sania Herbert	92	AI & DS
ATT002	23-AIDS-002	Rahul Kumar	85	AI & DS
ATT003	23AIDS003	Priya Nair	95	CSE
ATT004	23AIDS004	Arun Raj	78	ECE
ATT005	23AIDS005	Meena Joseph	88	AI and Data Science
ATT006	23AIDS006	Vikram Singh	80	CSE

```python
import pandas as pd
import numpy as np
import re
from rapidfuzz import fuzz
# ------------------------------------------------------------
# 1. LOAD DATA
# ------------------------------------------------------------
file_path = r"C:\Users\DELL\Downloads\Q4_Student_Dataset.xlsx"
admission = pd.read_excel(file_path, sheet_name="Admission")
examination = pd.read_excel(file_path, sheet_name="Examination")
attendance = pd.read_excel(file_path, sheet_name="Attendance")
# ------------------------------------------------------------
# 2. STANDARDIZE COLUMN NAMES
# ------------------------------------------------------------
for df in [admission, examination, attendance]:
    df.columns = (
        df.columns
        .str.strip()
        .str.lower()
        .str.replace(" ", "_")
    )
# Rename columns
admission = admission.rename(columns={
    "student_name": "name",
    "register_number": "reg_no",
    "email_address": "email"
})
examination = examination.rename(columns={
    "student": "name",
    "registration_no": "reg_no",
    "email_id": "email"
})
attendance = attendance.rename(columns={
    "student_name": "name",
    "registration_number": "reg_no"
})
```

```
})
# ------------------------------------------------------------
# 3. CLEAN NAME
# ------------------------------------------------------------
def clean_name(name):
    if pd.isna(name):
        return np.nan
    name = str(name).strip()
    name = re.sub(r"[^A-Za-z ]", "", name)
    name = re.sub(r"\s+", " ", name)
    return name.title()
# ------------------------------------------------------------
# 4. REGEX EMAIL VALIDATION
# ------------------------------------------------------------
def clean_email(email):
    if pd.isna(email):
        return np.nan
    email = str(email).strip().lower()
    pattern = r"^[\w\.-]+@[\w\.-]+\.\w+$"
    pattern = r"^[\w\.-]+@[\w\.-]+\.\w+$"
    if re.match(pattern, email):
        return email
    return np.nan
# ------------------------------------------------------------
# 5. REGISTER NUMBER STANDARDIZATION
# ------------------------------------------------------------
def clean_reg_no(reg_no):
    if pd.isna(reg_no):
        return np.nan
    reg_no = str(reg_no).upper()
    reg_no = re.sub(
        r"[^A-Z0-9]",
        "",
        reg_no
    )
    return reg_no
# ------------------------------------------------------------
# 6. CLEAN DATASETS
# ------------------------------------------------------------
for df in [admission, examination, attendance]:
    if "name" in df.columns:
        df["name"] = df["name"].apply(clean_name)
    if "reg_no" in df.columns:
        df["reg_no"] = df["reg_no"].apply(clean_reg_no)
    if "email" in df.columns:
        df["email"] = df["email"].apply(clean_email)
# ------------------------------------------------------------
# 7. REMOVE DUPLICATES
# ------------------------------------------------------------
admission = admission.drop_duplicates()
examination = examination.drop_duplicates()
attendance = attendance.drop_duplicates()
# ------------------------------------------------------------
# 8. MERGE DATASETS
# ------------------------------------------------------------
```

```python
# ------------------------------------------------------------
# 8. MERGE DATASETS
# ------------------------------------------------------------
students = pd.merge(
    admission,
    examination,
    on="reg_no",
    how="outer",
    suffixes=("_admission", "_exam")
)
students = pd.merge(
    students,
    attendance,
    on="reg_no",
    how="outer",
    suffixes=("", "_attendance")
)
# ------------------------------------------------------------
# 9. FUZZY NAME MATCHING
```

```
# ------------------------------------------------------
# 9. FUZZY NAME MATCHING
# ------------------------------------------------------
if "name_admission" in students.columns and \
   "name_exam" in students.columns:
    students["name_similarity"] = students.apply(
        lambda row: fuzz.ratio(
            str(row["name_admission"]),
            str(row["name_exam"])
        ),
        axis=1
    )
# ------------------------------------------------------
# 10. NORMALIZATION OF MARKS
# ------------------------------------------------------
if "marks" in students.columns:
    students["marks"] = pd.to_numeric(
        students["marks"],
        errors="coerce"
    )
    min_marks = students["marks"].min()
    max_marks = students["marks"].max()
    students["marks_normalized"] = (
        (students["marks"] - min_marks) /
        (max_marks - min_marks)
    )
# ------------------------------------------------------
# 11. HANDLE MISSING VALUES
# ------------------------------------------------------
if "email" in students.columns:
    students["email"] = students["email"].fillna(
        "Not Available"
    )
if "department" in students.columns:
    students["department"] = students[
        "department"
# ------------------------------------------------------
# 12. SAVE MASTER DATASET
# ------------------------------------------------------
students.to_csv(
    "student_master.csv",
    index=False
)
students.to_json(
    "student_master.json",
    orient="records",
    indent=4
)
print("Student integration completed.")
print("Final shape:", students.shape)
print(students.head())
new_students = pd.DataFrame({
    "reg_no": ["23AIDS101"],
    "name": ["Sania Herbert"],
    "email": ["sania.herbert@gmail.com"],
    "department": ["AI & DS"],
    "marks": [85]
})
new_students["reg_no"] = new_students[
    "reg_no"
].apply(clean_reg_no)
new_students["name"] = new_students[
    "name"
].apply(clean_name)
new_students["email"] = new_students[
    "email"
].apply(clean_email)
print("\nCleaned New Student:")
print(new_students)
```

OUTPUT:

Student integration completed.
Final shape: (6, 17)

	admission_id	reg_no	name_admission	email_admission	...	attendance_percent	department_name	name_similarity	marks_normalized
0	ADM001	23AIDS001	Sania Herbert	sania.herbert@gmail.com	...	92	AI & DS	100.000000	0.875000
1	ADM002	23AIDS002	Rahul Kumar	rahul.kumar@gmail.com	...	85	AI & DS	100.000000	0.375000
2	ADM003	23AIDS003	Priya Nair	priya.nair@gmail.com	...	95	CSE	100.000000	1.000000
3	ADM004	23AIDS004	Arun Raj	arun.raj@gmail.com	...	78	ECE	94.117647	0.000000
4	ADM005	23AIDS005	Meena Joseph	meena.joseph@gmail.com	...	88	AI and Data Science	100.000000	0.708333

[5 rows x 17 columns]

Cleaned New Student:

	reg_no	name	email	department	marks
0	23AIDS101	Sania Herbert	sania.herbert@gmail.com	AI & DS	85